CHATTIE · BLOG

Plano de Implementação v3

Cenário 1: MDX + Outstatic + Claude Code

Estratégia Bilíngue: PT-BR primeiro → EN estruturado para o futuro

Domínio **trychattie.com**	Blog PT-BR **trychattie.com/pt-br/blog**	Blog EN (futuro) **trychattie.com/blog**
CMS **MDX + Outstatic**	Motor agêntico **Claude Code + MCPs**	Meta **1 post/dia PT-BR**

Estratégia Bilíngue: Por que PT-BR Primeiro

A decisão de começar pelo PT-BR não é apenas operacional — é estratégica. Documentada aqui para referência futura.

Critério	
ICP primário do Chattie	Founders B2B brasileiros — o mercado que paga hoje
Volume editorial	1 post/dia num idioma vs. dividir energia em dois
SEO: autoridade tópica	Profundidade em PT-BR antes de fragmentar
GEO: citação por LLMs	Claude/ChatGPT no BR indexam PT-BR como autoridade
Custo de reverter	Zero — estrutura já suporta EN como rota adicional
Tempo até 1º post live	5–8h (vs. 10–14h para bilíngue simultâneo)
Quando adicionar EN	Após 30 posts PT-BR + processo validado (~30 dias)
Estratégia de conteúdo EN	Tradução + adaptação dos melhores posts PT-BR via Claude Code (revisão obrigatória)

⚠ POR QUE NÃO TRADUÇÃO AUTOMÁTICA COMO ESTRATÉGIA PRINCIPAL

A estratégia de tradução (Opção C) foi descartada como método principal porque: (1) Google detecta e desvaloriza conteúdo que é claramente tradução automática sem adaptação editorial; (2) LLMs também detectam padrões de tradução e reduzem a probabilidade de citação. A tradução via Claude Code é usada como ACELERADOR do blog EN — não como método de criação. Todo post traduzido exige revisão e adaptação de contexto (ex: substituir exemplos BR por exemplos globais).

Arquitetura de URLs — Espelhando o Site Atual

A arquitetura do blog espelha exatamente a estrutura do site HTML existente. Sem surpresas, sem exceções.

 # Mapa completo de URLs — presente e futuro

```
# Estrutura atual do site (não muda)
trychattie.com/           → index.html (EN)
trychattie.com/pt-br/     → pt-br/index.html (PT-BR)

# middleware.js (não muda)
# visitante BR na raiz → redireciona para /pt-br/
# visitante não-BR em /pt-br/ → redireciona para /
# cookie chattie-lang preserva escolha manual

# Nova estrutura do blog (projeto Next.js separado)
trychattie.com/pt-br/blog/          → listagem PT-BR   ← FASE 1 (agora)
trychattie.com/pt-br/blog/[slug]/   → post PT-BR       ← FASE 1 (agora)
trychattie.com/blog/                → listagem EN      ← FASE 6 (futuro, ~30 dias)
trychattie.com/blog/[slug]/         → post EN          ← FASE 6 (futuro, ~30 dias)

# Sitemaps separados
trychattie.com/sitemap-pt-br.xml    ← gerado pelo Next.js (PT-BR)
trychattie.com/sitemap-en.xml       ← gerado pelo Next.js (EN, fase futura)

# hreflang (quando EN existir)
# <link rel="alternate" hreflang="pt-BR" href="trychattie.com/pt-br/blog/[slug]"/>
# <link rel="alternate" hreflang="en"    href="trychattie.com/blog/[slug]"/>
```

✅ PREPARAR AMBOS OS REWRITES DESDE O DIA 1

O vercel.json do projeto HTML existente precisa de DOIS rewrites: /pt-br/blog/:path* → chattie-blog.vercel.app/pt-br/blog/:path* /blog/:path* → chattie-blog.vercel.app/blog/:path* (preparar agora mesmo sem conteúdo EN ainda). Preparar os dois agora evita mexer no vercel.json do site principal no futuro.

Estrutura do Projeto Next.js

O projeto chattie-blog é estruturado para suportar PT-BR agora e EN no futuro sem refactor.

```
# Estrutura de pastas — chattie-blog/
chattie-blog/
├── app/
│   ├── pt-br/
│   │   └── blog/
│   │       ├── page.tsx              # listagem PT-BR
│   │       └── [slug]/
│   │           └── page.tsx          # post PT-BR
│   ├── blog/                         # preparar pasta (vazia até Fase 6)
│   │   ├── page.tsx                  # listagem EN (placeholder)
│   │   └── [slug]/
│   │       └── page.tsx              # post EN (placeholder)
│   ├── sitemap-pt-br.ts              # sitemap PT-BR
│   ├── sitemap-en.ts                 # sitemap EN (preparar vazio)
│   └── robots.ts                     # robots.txt unificado
│
├── content/
│   ├── blog/                         # posts PT-BR (.mdx)
│   │   └── como-founders-b2b-vendem-pelo-linkedin.mdx
│   └── blog-en/                      # posts EN (.mdx) — vazio até Fase 6
│
├── lib/
│   ├── posts-pt.ts                   # getAllPosts() e getPostBySlug() PT-BR
│   └── posts-en.ts                   # idem para EN (preparar vazio)
│
├── components/
│   ├── mdx-components.tsx            # componentes MDX compartilhados
│   └── json-ld.tsx                   # Article schema (compartilhado)
│
├── docs/
│   ├── post-schema-pt.md             # frontmatter padrão PT-BR
│   └── post-schema-en.md             # frontmatter padrão EN (para futuro)
│
├── CLAUDE.md                         # instruções do agente
├── next.config.ts
├── package.json
└── vercel.json                       # não existe (rewrites ficam no site principal)
```

> 💡 **CRIAR AS PASTAS EN AGORA — MESMO VAZIAS**
> As pastas /app/blog/ e /content/blog-en/ são criadas agora mas ficam com placeholders. Isso garante que o projeto já "conhece" a rota EN — sem refactor quando o momento chegar. O custo de criar as pastas vazias agora é zero. O custo de refatorar depois é alto.

Configuração do vercel.json — Site Existente

Única modificação no site HTML atual. Dois rewrites — ambos configurados desde o Dia 1.

```
// vercel.json — adicionar ao projeto do site HTML
// vercel.json no projeto trychattie-site (site HTML existente)
//
// ATENÇÃO: verificar se já existe um vercel.json.
// Se existir, adicionar o bloco "rewrites" dentro do JSON existente.
// Se não existir, criar com este conteúdo:
{
  "rewrites": [
    {
      "source": "/pt-br/blog/:path*",
      "destination": "https://chattie-blog.vercel.app/pt-br/blog/:path*"
    },
    {
      "source": "/blog/:path*",
      "destination": "https://chattie-blog.vercel.app/blog/:path*"
    }
  ]
}

// TESTE após deploy:
// trychattie.com/pt-br/blog → deve responder (mesmo sem posts ainda)
// trychattie.com/blog → deve responder (placeholder EN)
//
// O middleware.js de geo-redirect NÃO conflita com rewrites.
// Rewrites no vercel.json são processados ANTES do middleware.
// Visitante BR acessa /pt-br/blog diretamente — sem redirect intermediário.
```

Frontmatter Padrão — PT-BR e EN

Dois schemas de frontmatter: um por idioma. Campos diferentes para SEO localizado.

PT-BR — /docs/post-schema-pt.md

```
# Frontmatter PT-BR — /docs/post-schema-pt.md
---
title: "Como founders B2B fecham mais pelo LinkedIn em 2025"
slug: "founders-b2b-linkedin-2025"
lang: "pt-BR"
date: "2025-04-03"
publishedAt: "2025-04-03T09:00:00-03:00"
description: "Meta description SEO em PT-BR — 140-160 caracteres com keyword principal"
excerpt: "Resumo para card de listagem — 1-2 frases em PT-BR"
category: "social-selling"
tags: ["linkedin", "b2b", "vendas", "founder-led-sales", "chattie"]
image: "/pt-br/blog/og/founders-linkedin-2025.jpg"
author: "Thiago Lisboa"
readTime: "8 min"
canonicalUrl: "https://trychattie.com/pt-br/blog/founders-b2b-linkedin-2025"
structuredData: "article"   # article | howto | faq
geoOptimized: true
```

```
# hreflang para quando o post EN existir (deixar vazio por enquanto):
enSlug: ""   # "how-b2b-founders-close-more-linkedin-2025" quando traduzido
---
```

EN — /docs/post-schema-en.md (para Fase 6)

```
# Frontmatter EN — /docs/post-schema-en.md
---
title: "How B2B Founders Close More Deals via LinkedIn in 2025"
slug: "how-b2b-founders-close-more-linkedin-2025"
lang: "en"
date: "2025-05-01"
publishedAt: "2025-05-01T09:00:00-03:00"
description: "SEO meta description in English — 140-160 chars with primary keyword"
excerpt: "One or two sentences for listing card"
category: "social-selling"
tags: ["linkedin", "b2b", "sales", "founder-led-sales", "chattie"]
image: "/blog/og/how-b2b-founders-close-more-linkedin-2025.jpg"
author: "Thiago Lisboa"
readTime: "8 min"
canonicalUrl: "https://trychattie.com/blog/how-b2b-founders-close-more-linkedin-2025"
structuredData: "article"
geoOptimized: true
# hreflang para versão PT-BR:
ptSlug: "founders-b2b-linkedin-2025"
---
```

Plano de Implementação — Fase a Fase

FASE 1

Setup do Projeto Next.js + Vercel

⏱ 1–2 horas

01

Criar o projeto Next.js chattie-blog

Scaffolding com App Router, TypeScript e Tailwind.

- npx create-next-app@latest chattie-blog --typescript --tailwind --app --src-dir=false --eslint
- Criar estrutura de pastas: /content/blog, /content/blog-en, /docs
- Criar /app/blog/ com placeholder (página vazia) para rota EN futura

› Criar /app/pt-br/blog/ para rota PT-BR ativa

02

Deploy no Vercel

Publicar o projeto vazio para obter a URL interna.

- Criar repositório GitHub: chattie-blog
- Vercel: New Project → Import → chattie-blog
- Deploy automático → URL interna: chattie-blog.vercel.app
- Confirmar que a URL responde antes do próximo passo

03

Configurar vercel.json no site HTML existente

Dois rewrites: /pt-br/blog/* e /blog/* — ambos agora.

- No repo trychattie-site: verificar se vercel.json já existe (cat vercel.json)
- Adicionar os dois blocos de rewrite conforme seção anterior
- git commit -m "feat: add blog rewrites" && git push
- Aguardar deploy do Vercel (~30s) e testar ambas as URLs
- TESTE: trychattie.com/pt-br/blog → deve retornar algo do Next.js
- TESTE: middleware.js ainda funciona (visitar trychattie.com no Brasil)

FASE 2

Blog PT-BR: Rotas + Outstatic

⏱ **2–3 horas**

04

Instalar e configurar Outstatic

CMS leve para edição editorial sem terminal.

- npm install outstatic
- Criar GitHub OAuth App: github.com/settings/developers → New OAuth App
- Homepage URL: https://trychattie.com
- Callback URL: https://chattie-blog.vercel.app/api/outstatic/callback
- Adicionar env vars no Vercel: GITHUB_ID, GITHUB_SECRET, OST_TOKEN_SECRET
- Criar /app/outstatic/[[...ost]]/page.tsx e /app/api/outstatic/[...outstatic]/route.ts
- Configurar Outstatic para usar /content/blog como pasta de posts PT-BR

05

Criar lib/posts-pt.ts

Funções de leitura dos posts PT-BR do filesystem.

- npm install gray-matter next-mdx-remote
- Criar /lib/posts-pt.ts com getAllPostsPt() e getPostBySlugPt()

- Os posts ficam em /content/blog/*.mdx
- Retornar: title, slug, date, description, excerpt, category, tags, image, lang

// /lib/posts-pt.ts

```
// /lib/posts-pt.ts
import fs from 'fs'
import path from 'path'
import matter from 'gray-matter'

const DIR = path.join(process.cwd(), 'content/blog')

export function getAllPostsPt() {
  if (!fs.existsSync(DIR)) return []
  return fs.readdirSync(DIR)
    .filter(f => f.endsWith('.mdx'))
    .map(file => {
      const { data } = matter(fs.readFileSync(path.join(DIR, file), 'utf-8'))
      return { ...data, slug: data.slug || file.replace(".mdx","") }
    })
    .sort((a,b) => new Date(b.date) - new Date(a.date))
}

export function getPostBySlugPt(slug: string) {
  const file = path.join(DIR, `${slug}.mdx`)
  return matter(fs.readFileSync(file, 'utf-8'))
}
```

06 Criar rotas /pt-br/blog e /pt-br/blog/[slug]

Listagem e post individual PT-BR.

- Criar /app/pt-br/blog/page.tsx — listagem com getAllPostsPt()
- Criar /app/pt-br/blog/[slug]/page.tsx — post com getPostBySlugPt()
- Implementar generateStaticParams() para SSG
- Criar /components/mdx-components.tsx — customizar h1, h2, a, img, code
- Criar /components/blog-card.tsx — card de post para a listagem

FASE 3 SEO Técnico PT-BR Completo

⏱ **2–3 horas**

07 generateMetadata() PT-BR

Meta tags, Open Graph e canonical URL em PT-BR.

- Em /app/pt-br/blog/[slug]/page.tsx: implementar generateMetadata({ params })
- title: frontmatter.title | description: frontmatter.description
- canonical: frontmatter.canonicalUrl (https://trychattie.com/pt-br/blog/[slug])
- og:locale: "pt_BR" | og:type: "article"
- article:published_time, article:author, article:tag
- Preparar hreflang vazio — será preenchido quando EN existir:
- alternates: { languages: { "en": undefined } } // placeholder

08 Sitemap PT-BR dinâmico

Gerar sitemap-pt-br.xml com todos os posts publicados.

- Criar /app/sitemap-pt-br.ts (Next.js serve como /sitemap-pt-br.xml)
- Incluir: trychattie.com/pt-br/blog + todos os slugs PT-BR
- Registrar no Google Search Console em: trychattie.com
- Adicionar propriedade separada para pt-br se necessário

// /app/sitemap-pt-br.ts

```
// /app/sitemap-pt-br.ts
import { getAllPostsPt } from '@/lib/posts-pt'

export default function sitemapPt() {
  const posts = getAllPostsPt()
  return [
    {
      url: "https://trychattie.com/pt-br/blog",
      lastModified: new Date(),
      changeFrequency: "daily",
      priority: 0.9,
    },
    ...posts.map(post => ({
      url: `https://trychattie.com/pt-br/blog/${post.slug}`,
      lastModified: new Date(post.date),
      changeFrequency: "weekly",
      priority: 0.8,
    }))
  ]
}
```

09 robots.ts — Permitir AI crawlers

Crítico para GEO. Nunca bloquear GPTBot, ClaudeBot, PerplexityBot.

- Criar /app/robots.ts (Next.js serve como /robots.txt)
- Permitir explicitamente: Googlebot, GPTBot, ClaudeBot, CCBot, PerplexityBot
- Referenciar ambos os sitemaps (PT-BR agora, EN como placeholder)

// /app/robots.ts — permitir todos os AI crawlers

```
// /app/robots.ts
export default function robots() {
  return {
    rules: [
      { userAgent: "*",            allow: ["/pt-br/blog/", "/blog/"] },
      { userAgent: "GPTBot",       allow: ["/pt-br/blog/", "/blog/"] },
      { userAgent: "ClaudeBot",    allow: ["/pt-br/blog/", "/blog/"] },
      { userAgent: "CCBot",        allow: ["/pt-br/blog/", "/blog/"] },
      { userAgent: "PerplexityBot",allow: ["/pt-br/blog/", "/blog/"] },
    ],
    sitemap: [
      "https://trychattie.com/sitemap-pt-br.xml",
      // "https://trychattie.com/sitemap-en.xml",  // descomentar na Fase 6
    ],
  }
}
```

10

JSON-LD Article schema PT-BR

Schema markup para Google e LLMs — fator mais crítico para GEO.

- Criar /components/json-ld.tsx — componente ArticleJsonLd
- Campos: headline, description, author, datePublished, dateModified, image, publisher
- inLanguage: "pt-BR"
- Incluir em /app/pt-br/blog/[slug]/page.tsx
- Preparar suporte a FAQPage schema para posts com structuredData == "faq"

// /components/json-ld.tsx — compartilhado PT-BR e EN

```
// /components/json-ld.tsx
export function ArticleJsonLd({ post, lang = "pt-BR" }) {
  const schema = {
    "@context": "https://schema.org",
    "@type": "Article",
    "headline": post.title,
    "description": post.description,
    "inLanguage": lang,
    "datePublished": post.publishedAt,
    "dateModified": post.date,
    "author": {
      "@type": "Person",
      "name": "Thiago Lisboa",
      "url": "https://trychattie.com"
    },
    "publisher": {
      "@type": "Organization",
      "name": "Chattie",
      "url": "https://trychattie.com"
    },
    "image": { "@type": "ImageObject", "url": post.image, "width": 1200, "height": 630 },
    "mainEntityOfPage": { "@type": "WebPage", "@id": post.canonicalUrl },
  }
  return <script type="application/ld+json"
```

```
    dangerouslySetInnerHTML={{ __html: JSON.stringify(schema) }} />
}

// Uso em /app/pt-br/blog/[slug]/page.tsx:
// <ArticleJsonLd post={post} lang="pt-BR" />
// Reutilizar em EN: <ArticleJsonLd post={post} lang="en" />
```

FASE 4

Migração do CSV — Histórico PT-BR

⏱ **1–2 horas**

1 1

Script de migração CSV → MDX PT-BR

Claude Code escreve e roda o script.

- npm install turndown
- Prompt para Claude Code:
- "Escreva um script Node.js que:
- 1. Leia blog-history.csv (colunas: title, content)
- 2. Converta HTML para Markdown com turndown
- 3. Gere slug a partir do title (lowercase, hifens, sem acentos)
- 4. Escreva /content/blog/[slug].mdx com o frontmatter de /docs/post-schema-pt.md
- 5. Preencha description com os primeiros 155 chars do texto limpo
- 6. Preencha excerpt com os primeiros 100 chars
- 7. lang: pt-BR, author: Thiago Lisboa, geoOptimized: false"
- Revisar 5 posts gerados como amostra
- node scripts/migrate-csv-pt.js
- git add content/blog/ && git commit -m "chore: migrate PT-BR CSV history"

💡 **OTIMIZAÇÃO EM MASSA PÓS-MIGRAÇÃO VIA CLAUDE CODE**

Após a migração, rodar uma passagem de otimização de frontmatter via Claude Code:claude "Revise os frontmatters de todos os posts em /content/blog/ e melhore: - description: 140-160 chars, incluir keyword principal em PT-BR - excerpt: 100 chars, atrativo para leitura - tags: adicionar 3-5 tags relevantes - category: classificar em uma das categorias: social-selling, linkedin, b2b, ia-para-vendas, chattie"Isso transforma a migração em otimização SEO em massa automatizada.

FASE 5

CLAUDE.md + MCPs + Workflow Agêntico PT-BR

⏱ 2–3 horas

1 2

Criar CLAUDE.md

Arquivo de instruções lido automaticamente pelo Claude Code. Define tudo.

- Criar na raiz de chattie-blog/
- Definir: onde salvar posts PT-BR (/content/blog/)
- Definir: frontmatter schema de /docs/post-schema-pt.md
- Definir: regras de SEO PT-BR (keyword em PT, description em PT)
- Definir: regras de GEO (answer-first, definições, dados com fonte)
- Definir: tom de voz (direto, técnico, founders B2B brasileiros)
- Definir: ferramentas MCP disponíveis

 # CLAUDE.md — colar na raiz do projeto

```
# CLAUDE.md — Agente de Conteúdo do Chattie

## Identidade e contexto
Blog do Chattie — AI SDR para LinkedIn.
ICP: founders, consultores e operadores B2B brasileiros que vendem pelo LinkedIn.
Tom: direto, técnico, sem floreio, sem motivação vazia. Sempre PT-BR.

## Criar um post PT-BR
Salvar em: /content/blog/[slug].mdx
Seguir EXATAMENTE o frontmatter em /docs/post-schema-pt.md
lang: "pt-BR" obrigatório em todo post
Tamanho mínimo: 1800 palavras
Estrutura de H2: usar perguntas reais do ICP ("Como founders B2B usam X?")
Começar cada seção com resposta direta (answer-first)
Incluir 1 definição explícita por conceito novo
Incluir listas estruturadas — não só prosa
Incluir 2-3 internal links para outros posts em /pt-br/blog/
Incluir FAQ ao final (mínimo 3 perguntas)

## SEO PT-BR obrigatório
description: 140-160 chars, keyword principal em PT-BR
slug: lowercase, hifens, sem acentos (ex: como-usar-linkedin-para-b2b)
canonicalUrl: https://trychattie.com/pt-br/blog/[slug]

## GEO obrigatório
Citar dados com fonte e data: "Segundo [fonte], em [ano]..."
geoOptimized: true quando o post define conceitos, cita dados, tem listas
structuredData: "faq" se o post tem seção FAQ com 3+ perguntas
```

```
## Ferramentas MCP disponíveis
brave-search: pesquisar SERP antes de criar cada post
filesystem: ler/escrever arquivos em /content/blog/

## Para criar um post EN (Fase 6 — não usar ainda)
Salvar em: /content/blog-en/[slug].mdx
Seguir /docs/post-schema-en.md
lang: "en" obrigatório
```

1
3

Configurar Brave Search MCP

Análise de SERP e concorrentes direto do terminal.

- Obter API key em: search.brave.com/register (plano Free: 2000 queries/mês)
- npm install -g @modelcontextprotocol/server-brave-search
- Adicionar em ~/.claude/config.json (ver código abaixo)
- Testar: claude "Use o Brave Search para listar os top 5 resultados de social selling B2B no Brasil"

// ~/.claude/config.json

```
// ~/.claude/config.json — configuração global do Claude Code
{
  "mcpServers": {
    "brave-search": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-brave-search"],
      "env": { "BRAVE_API_KEY": "sua-api-key-aqui" }
    },
    "filesystem": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-filesystem",
               "/caminho/absoluto/para/chattie-blog"]
    }
  }
}
```

1
4

Workflow diário PT-BR

O processo de 15-20 min do terminal ao post publicado.

 # Workflow diário — do terminal ao post publicado

```
# ═══ WORKFLOW DIÁRIO PT-BR — 15-20 min do terminal ao post live ═══

# ETAPA 1: Pesquisa de SERP (2 min)
claude "Use o Brave Search MCP para pesquisar: [KEYWORD EM PT-BR].
       Analise os top 5 resultados em PT-BR e me dê:
       1. Word count médio dos resultados
       2. H2s mais comuns
       3. Perguntas que nenhum resultado responde bem
```

```
        4. Ângulo diferente que posso explorar para o ICP do Chattie"

# ETAPA 2: Geração do post (2 min de prompt)
claude "Com base na análise acima, crie um post completo em PT-BR sobre [TEMA].
       Keyword principal: [KEYWORD].
       ICP: founders e consultores B2B brasileiros que vendem pelo LinkedIn.
       Mínimo 1800 palavras. Incluir: definições, dados com fonte, listas,
       2-3 links internos para /pt-br/blog/, FAQ ao final.
       Salve em /content/blog/[slug].mdx com frontmatter completo."

# ETAPA 3: Revisão (5-10 min)
code content/blog/[slug].mdx
# Verificar: factual accuracy, tom, links internos, slug, imagem OG

# ETAPA 4: Deploy (30 segundos)
git add content/blog/[slug].mdx
git commit -m "post(pt-br): [titulo-resumido]"
git push
# Vercel detecta o push → deploya em ~60s

# ETAPA 5: Verificação (1 min)
# Acessar: trychattie.com/pt-br/blog/[slug]
# Verificar JSON-LD: curl -s https://trychattie.com/pt-br/blog/[slug] | grep "ld+json"
# Verificar sitemap: https://trychattie.com/sitemap-pt-br.xml
```

Fase 6 (Futuro): Ativação do Blog EN

A ativar após ~30 posts PT-BR publicados e processo validado. Estimativa: +1 sprint de 4–6 horas.

Passo	
6.1 — Criar lib/posts-en.ts	Clonar posts-pt.ts apontando para /content/blog-en/. Função getAllPostsEn() e getPostBySlugEn().
6.2 — Ativar rotas /blog e /blog/[slug]	Substituir o placeholder pelo componente real usando getAllPostsEn(). generateMetadata() com og:locale:"en_US".
6.3 — Criar sitemap-en.ts	Clonar sitemap-pt-br.ts apontando para os posts EN. Descomentar a linha do sitemap EN no robots.ts.
6.4 — Adicionar hreflang	Em cada post PT-BR com enSlug preenchido: alternates.languages.en = URL EN. Em cada post EN: alternates.languages["pt-BR"] = URL PT-BR.
6.5 — Selecionar posts para traduzir	Escolher os 10 posts PT-BR de melhor desempenho (GSC: cliques + impressões). Traduzir + adaptar via Claude Code.
6.6 — Atualizar CLAUDE.md	Habilitar a seção "Para criar um post EN". Definir slug em inglês, canonicalUrl EN, og:locale en_US.

6.7 — Atualizar robots.ts	Descomentar a linha do sitemap-en.xml. Registrar no Google Search Console (propriedade EN).

🌐 **ESTRATÉGIA DE CONTEÚDO EN**

Estratégia de conteúdo EN: usar os melhores posts PT-BR como base, traduzir + adaptar com Claude Code. "Adaptar" significa: (1) substituir exemplos BR por exemplos globais, (2) ajustar referências culturais, (3) verificar se dados e fontes citados têm versão em inglês, (4) revisar tom para audiência global. Meta para o blog EN: publicar 3 posts/semana no início (vs. 1/dia no PT-BR). Velocidade menor, qualidade editorial maior — audiência EN é mais exigente.

Timeline: 7 Dias para o Primeiro Post PT-BR Agêntico

DIA 1	Criar projeto Next.js. Deploy no Vercel. Configurar os dois rewrites no vercel.json do site HTML. Confirmar trychattie.com/pt-br/blog responde.
DIA 2	Instalar Outstatic. Criar lib/posts-pt.ts. Implementar rotas /pt-br/blog e /pt-br/blog/[slug] básicas. Criar pasta /content/blog com 1 post de teste.
DIA 3	SEO técnico PT-BR: generateMetadata(), sitemap-pt-br.ts, robots.ts (com AI crawlers), json-ld.tsx. Testar com post de exemplo.
DIA 4	Rodar script de migração CSV → /content/blog/. Revisar 5 posts amostra. Commit do histórico completo. Verificar posts no /pt-br/blog.
DIA 5	Criar CLAUDE.md. Configurar Brave Search MCP. Testar workflow completo: SERP → gerar → revisão → git push → verificar live.
DIA 6	1º post agêntico real: pesquisar SERP → gerar → revisar → publicar. Registrar sitemap-pt-br.xml no Google Search Console.
DIA 7	Ajustes no CLAUDE.md com base no post do dia 6. Publicar 2º post. Processo validado. Modo operação: 1 post/dia.

⏱ **Total PT-BR: 10–14 horas · 7 dias · 1º post agêntico live no Dia 6**

⏱ **Total EN (Fase 6, futuro): +4–6 horas · 1 sprint adicional · ~30 dias depois**

Resumo de Esforço por Fase

Fase	Entrega	Horas
1	Setup Next.js + Vercel deploy + rewrites no site HTML	**1–2h**
2	Blog PT-BR: Outstatic + lib/posts-pt.ts + rotas funcionais	**2–3h**
3	SEO técnico PT-BR: metadata, sitemap, robots, JSON-LD	**2–3h**
4	Migração CSV → /content/blog/ com frontmatter otimizado	**1–2h**
5	CLAUDE.md + Brave Search MCP + workflow agêntico validado	**2–3h**
—	**TOTAL PT-BR (Fases 1–5)**	**8–13h**
6	Ativação EN: rotas, lib, sitemap, hreflang, tradução de 10 posts	**4–6h**
—	**TOTAL COMPLETO (Fases 1–6)**	**12–19h**

Próximo Passo: Agora

Abra dois terminais em paralelo:

Terminal 1 — repo do site HTML:

```
cat vercel.json
```

Se não existir, criar com os dois rewrites da Fase 1.

Terminal 2 — nova pasta:

```
npx create-next-app@latest chattie-blog --typescript --tailwind --app
```

Esses dois comandos são o Dia 1 completo.